Assignment 3: Network Monitoring, Intrusion Detection and Defense

Name & Student ID: [Your Name], [Your ID]

Part 1: Network Change Monitoring with Bash

Script Explanation

The script (`network_monitor.sh`) is organized into four functions called by `main`. `scan_network` creates the output directory and runs an nmap SYN scan across `192.168.10.0/24`, saving grepable output to a timestamped file. `parse_scan_results` uses `awk` and `sed` to extract IP-to-port mappings from the raw scan file and writes them to `current_state.txt`. `detect_changes` compares `current_state.txt` against `previous_state.txt` to identify new hosts, offline hosts, and per-host port opens/closes; on the first run it treats the scan as the baseline. `log_changes` writes timestamped entries to `log.txt` then copies `current_state.txt` to `previous_state.txt` to update the baseline for the next run.

Task 2: Cron Automation

Screenshots:

- [Insert screenshot of `sudo crontab -l` showing the scheduled entry]

```
┌──(kevin㉿kali)-[~/Documents/assignment3/scans]
└─$ crontab -l
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h  dom mon dow   command
*/10 * * * * /home/kevin/Documents/assignment3/network_monitor.sh
```

Task 3: Scenario Detection

Screenshots:

```
┌──(kevin㉿kali)-[~/Documents/assignment3/scans]
└─$ ls
current_state.txt   previous_state.txt        scan_2026-05-09_15-35.txt
network_monitor.sh  scan_2026-05-09_15-28.txt
```

- [Insert screenshot of `log.txt` showing VM-T detected as a new host]

```
- 192.168.10.13: Port 52530 closed
[2026-05-09 15:49] Change detected:

+ New host: 192.168.10.11
    Open ports: 8080
```

This screenshot shows both the new Host and Port 8080.

Scenario Summary

The monitoring script successfully detected every staged network event across the six steps. On the first run it logged the baseline with only VM-D visible on the subnet. When VM-T was powered on, the next scan caught it as a new host and listed all of its open ports, including FTP on 21 and HTTP on 80. VM-A was then detected the same way when it came online. Once a web server was started on VM-A, the script picked up

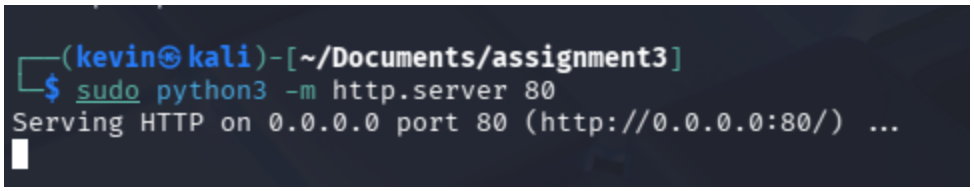
port 8080 opening on that host. The most significant detection came when the VSFTPD exploit was run from VM-A — the backdoor port 6200 appeared on VM-T within the next scan cycle, exactly as expected. Finally, when the web server on VM-A was stopped, the script correctly logged port 8080 as closed. The script had no false positives throughout the scenario and the log entries were clean and easy to read.

Part 2: Network Defense Using IPTables on VM-D

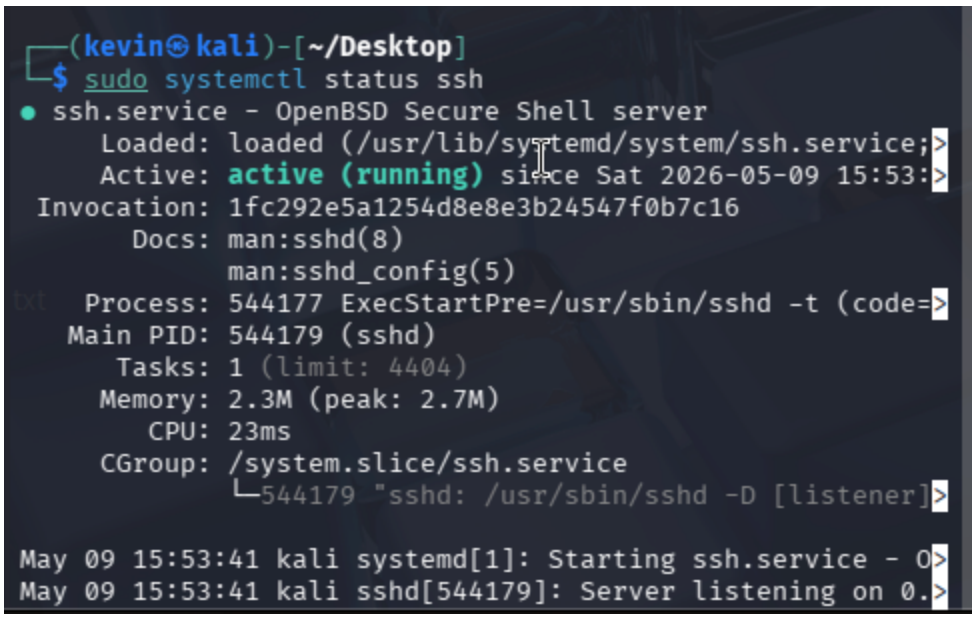
Task 1: Launch Services on VM-D

Screenshots:

- [Insert screenshot of the Python web server running on port 80]



- [Insert screenshot of `sudo systemctl status ssh` showing the service as active]



Task 2: Firewall Ruleset

Screenshots:

- [Insert screenshot of `sudo iptables -L -v -n` showing the full ruleset]

```
┌──(kevin㉿kali)-[~/Documents/assignment3]
└─$ cat rules.txt
# Generated by iptables-save v1.8.11 (nf_tables) on Sat May  9 16:04:05 2026
*filter
:INPUT DROP [0:0]
:FORWARD DROP [0:0]
:OUTPUT DROP [0:0]
:SYN_FLOOD - [0:0]
-A INPUT -i lo -j ACCEPT
-A INPUT -m conntrack --ctstate INVALID -j LOG --log-prefix "MALFORMED: "
-A INPUT -m conntrack --ctstate INVALID -j DROP
-A INPUT -m conntrack --ctstate RELATED,ESTABLISHED -j ACCEPT
-A INPUT -s 127.0.0.0/8 ! -i lo -j LOG --log-prefix "MALFORMED: "
-A INPUT -s 127.0.0.0/8 ! -i lo -j DROP
-A INPUT -s 192.168.10.12/32 -j LOG --log-prefix "MALFORMED: "
-A INPUT -s 192.168.10.12/32 -j DROP
-A INPUT ! -s 192.168.10.0/24 -m conntrack --ctstate NEW -j LOG --log-prefix
 "ILLEGAL-IP/PORT: "
-A INPUT ! -s 192.168.10.0/24 -m conntrack --ctstate NEW -j DROP
-A INPUT -p tcp -m tcp --tcp-flags FIN,SYN,RST,PSH,ACK,URG NONE -j LOG --log
-prefix "SCAN-TCP: "
-A INPUT -p tcp -m tcp --tcp-flags FIN,SYN,RST,PSH,ACK,URG NONE -j DROP
-A INPUT -p tcp -m tcp --tcp-flags FIN,SYN,RST,PSH,ACK,URG FIN,PSH,URG -j LO
G --log-prefix "SCAN-TCP: "
-A INPUT -p tcp -m tcp --tcp-flags FIN,SYN,RST,PSH,ACK,URG FIN,PSH,URG -j DR
OP
-A INPUT -p tcp -m tcp --tcp-flags FIN,SYN FIN,SYN -j LOG --log-prefix "SCAN
```

Task 3: Firewall Testing

Test A: ICMP Behavior

Screenshots:

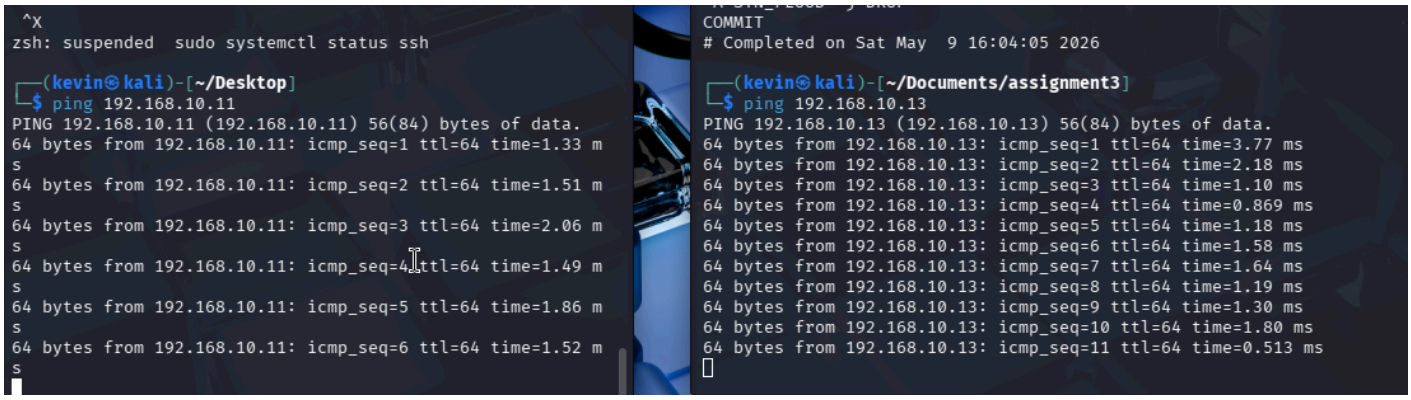
- [Insert screenshot of ping flood from VM-A to VM-D]

```
┌──(kevin㉿kali)-[~/Desktop]
└─$ ping 192.168.10.12
PING 192.168.10.12 (192.168.10.12) 56(84) bytes of data.
```

- [Insert screenshot of normal ping from VM-T to VM-D]

```
msfadmin@metasploitable:~$ ping 192.168.10.12
PING 192.168.10.12 (192.168.10.12) 56(84) bytes of data.
```

- [Insert screenshot of ping from VM-D to VM-A and VM-T]



Test B: TCP SYN Flood Detection

Screenshots:

- [Insert screenshot of SYN flood from VM-A with real IP]

```
┌──(kevin㉿kali)-[~/Desktop]
└─$ sudo hping3 -S --flood -p 80 192.168.10.12
[sudo] password for kevin:
HPING 192.168.10.12 (eth0 192.168.10.12): S set, 40 headers + 0 data bytes
hping in flood mode, no replies will be shown
^[[B^[[B^[[B^[[B^[[B^[[B^[[B^[[B^[[B^[[B^[[B^[[B^[[B^[[B^C
— 192.168.10.12 hping statistic —
10490148 packets transmitted, 0 packets received, 100% packet loss
round-trip min/avg/max = 0.0/0.0/0.0 ms

┌──(kevin㉿kali)-[~/Desktop]
└─$ sudo hping3 -S --flood -p -rand-source 192.168.10.12
HPING 192.168.10.12 (eth0 192.168.10.12): S set, 40 headers + 0 data bytes
hping in flood mode, no replies will be shown
```

```
┌──(kevin㉿kali)-[~/Documents/assignment3]
└─$ sudo dmesg | tail -30
[68664.187972] DOS-RATE-LIMIT: IN=eth0 OUT= MAC=ba:7b:37:f3:10:b6:22:fa:a1:9
c:78:62:08:00 SRC=192.168.10.11 DST=192.168.10.12 LEN=40 TOS=0x00 PREC=0x00
TTL=64 ID=37079 PROTO=TCP SPT=15509 DPT=0 WINDOW=512 RES=0x00 SYN URGP=0
[68664.190050] DOS-RATE-LIMIT: IN=eth0 OUT= MAC=ba:7b:37:f3:10:b6:22:fa:a1:9
c:78:62:08:00 SRC=192.168.10.11 DST=192.168.10.12 LEN=40 TOS=0x00 PREC=0x00
TTL=64 ID=38518 PROTO=TCP SPT=15510 DPT=0 WINDOW=512 RES=0x00 SYN URGP=0
[68664.190086] DOS-RATE-LIMIT: IN=eth0 OUT= MAC=ba:7b:37:f3:10:b6:22:fa:a1:9
c:78:62:08:00 SRC=192.168.10.11 DST=192.168.10.12 LEN=40 TOS=0x00 PREC=0x00
TTL=64 ID=52538 PROTO=TCP SPT=15511 DPT=0 WINDOW=512 RES=0x00 SYN URGP=0
[68664.190095] DOS-RATE-LIMIT: IN=eth0 OUT= MAC=ba:7b:37:f3:10:b6:22:fa:a1:9
c:78:62:08:00 SRC=192.168.10.11 DST=192.168.10.12 LEN=40 TOS=0x00 PREC=0x00
TTL=64 ID=29360 PROTO=TCP SPT=15512 DPT=0 WINDOW=512 RES=0x00 SYN URGP=0
[68664.190104] DOS-RATE-LIMIT: IN=eth0 OUT= MAC=ba:7b:37:f3:10:b6:22:fa:a1:9
c:78:62:08:00 SRC=192.168.10.11 DST=192.168.10.12 LEN=40 TOS=0x00 PREC=0x00
TTL=64 ID=62605 PROTO=TCP SPT=15513 DPT=0 WINDOW=512 RES=0x00 SYN URGP=0
[68664.190112] DOS-RATE-LIMIT: IN=eth0 OUT= MAC=ba:7b:37:f3:10:b6:22:fa:a1:9
c:78:62:08:00 SRC=192.168.10.11 DST=192.168.10.12 LEN=40 TOS=0x00 PREC=0x00
TTL=64 ID=54247 PROTO=TCP SPT=15514 DPT=0 WINDOW=512 RES=0x00 SYN URGP=0
[68664.190128] DOS-RATE-LIMIT: IN=eth0 OUT= MAC=ba:7b:37:f3:10:b6:22:fa:a1:9
c:78:62:08:00 SRC=192.168.10.11 DST=192.168.10.12 LEN=40 TOS=0x00 PREC=0x00
TTL=64 ID=43025 PROTO=TCP SPT=15515 DPT=0 WINDOW=512 RES=0x00 SYN URGP=0
```

Test C: Port Scan Resistance

Screenshots:

- [Insert screenshot of TCP Connect Scan (`nmap -sT`) results]

```
┌──(kevin㉿kali)-[~/Desktop]
└─$ sudo nmap -sT 192.168.10.12
Starting Nmap 7.98 ( https://nmap.org ) at 2026-05-09 16:12 -0600
Note: Host seems down. If it is really up, but blocking our ping probes, try -Pn
Nmap done: 1 IP address (0 hosts up) scanned in 1.50 seconds
```

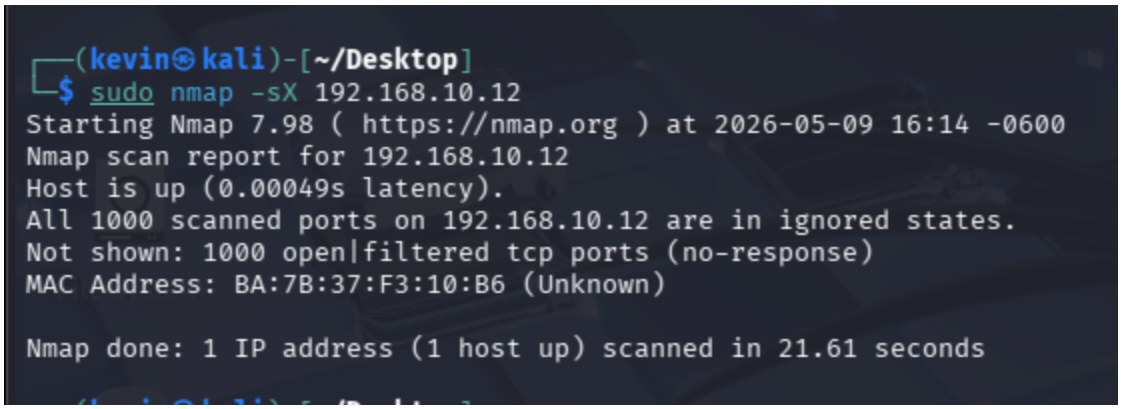
- [Insert screenshot of SYN Scan (`nmap -sS`) results]

```
┌──(kevin㉿kali)-[~/Desktop]
└─$ sudo nmap -sS 192.168.10.12
Starting Nmap 7.98 ( https://nmap.org ) at 2026-05-09 16:13 -0600
Nmap scan report for 192.168.10.12
Host is up (0.0011s latency).
Not shown: 999 filtered tcp ports (no-response)
PORT   STATE SERVICE
80/tcp open  http
MAC Address: BA:7B:37:F3:10:B6 (Unknown)

Nmap done: 1 IP address (1 host up) scanned in 13.98 seconds

┌──(kevin㉿kali)-[~/Desktop]
└─$
```

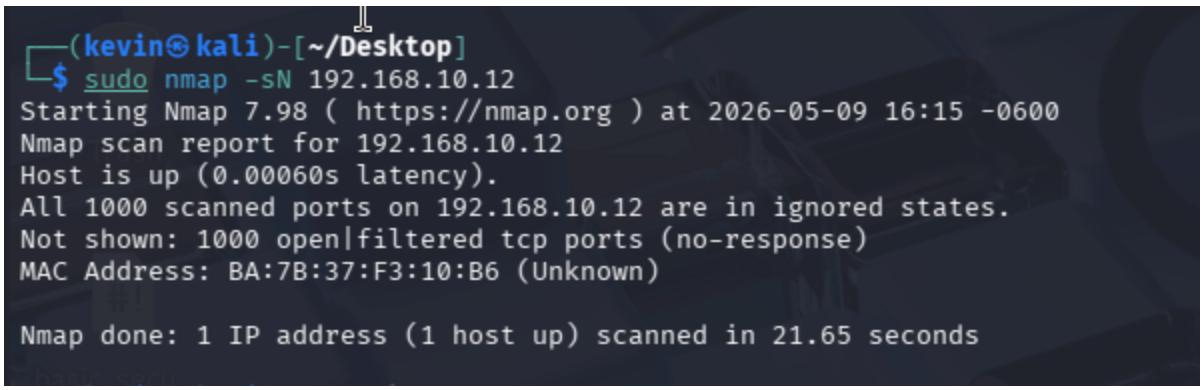
- [Insert screenshot of FIN Scan (`nmap -sF`) results]

```
┌──(kevin㉿kali)-[~/Desktop]
└─$ sudo nmap -sF 192.168.10.12
Starting Nmap 7.98 ( https://nmap.org ) at 2026-05-09 16:13 -0600
Nmap scan report for 192.168.10.12
Host is up (0.00044s latency).
All 1000 scanned ports on 192.168.10.12 are in ignored states.
Not shown: 1000 open|filtered tcp ports (no-response)
MAC Address: BA:7B:37:F3:10:B6 (Unknown)

Nmap done: 1 IP address (1 host up) scanned in 21.69 seconds

┌──(kevin㉿kali)-[~/Desktop]
└─$
```

- [Insert screenshot of Xmas Scan (`nmap -sX`) results]



- [Insert screenshot of NULL Scan (`nmap -sN`) results]



- [Insert screenshot of `dmesg` showing `SCAN-TCP` log entries]

```
┌──(kevin㉿kali)-[~/Documents/assignment3]
└─$ sudo dmesg | tail -30
[68892.118380] MALFORMED: IN=eth0 OUT= MAC=ba:7b:37:f3:10:b6:22:fa:a1:9c:78:
62:08:00 SRC=192.168.10.11 DST=192.168.10.12 LEN=40 TOS=0x00 PREC=0x00 TTL=5
4 ID=54585 PROTO=TCP SPT=64614 DPT=1277 WINDOW=1024 RES=0x00 URGP=0
[68892.118415] MALFORMED: IN=eth0 OUT= MAC=ba:7b:37:f3:10:b6:22:fa:a1:9c:78:
62:08:00 SRC=192.168.10.11 DST=192.168.10.12 LEN=40 TOS=0x00 PREC=0x00 TTL=4
3 ID=41446 PROTO=TCP SPT=64614 DPT=1812 WINDOW=1024 RES=0x00 URGP=0
[68892.118456] MALFORMED: IN=eth0 OUT= MAC=ba:7b:37:f3:10:b6:22:fa:a1:9c:78:
62:08:00 SRC=192.168.10.11 DST=192.168.10.12 LEN=40 TOS=0x00 PREC=0x00 TTL=3
9 ID=40893 PROTO=TCP SPT=64614 DPT=119 WINDOW=1024 RES=0x00 URGP=0
[68892.118474] MALFORMED: IN=eth0 OUT= MAC=ba:7b:37:f3:10:b6:22:fa:a1:9c:78:
62:08:00 SRC=192.168.10.11 DST=192.168.10.12 LEN=40 TOS=0x00 PREC=0x00 TTL=5
6 ID=5496 PROTO=TCP SPT=64614 DPT=6001 WINDOW=1024 RES=0x00 URGP=0
[68892.118521] MALFORMED: IN=eth0 OUT= MAC=ba:7b:37:f3:10:b6:22:fa:a1:9c:78:
62:08:00 SRC=192.168.10.11 DST=192.168.10.12 LEN=40 TOS=0x00 PREC=0x00 TTL=4
7 ID=24539 PROTO=TCP SPT=64614 DPT=163 WINDOW=1024 RES=0x00 URGP=0
[68892.118537] MALFORMED: IN=eth0 OUT= MAC=ba:7b:37:f3:10:b6:22:fa:a1:9c:78:
62:08:00 SRC=192.168.10.11 DST=192.168.10.12 LEN=40 TOS=0x00 PREC=0x00 TTL=5
7 ID=8377 PROTO=TCP SPT=64614 DPT=1198 WINDOW=1024 RES=0x00 URGP=0
[68892.118553] MALFORMED: IN=eth0 OUT= MAC=ba:7b:37:f3:10:b6:22:fa:a1:9c:78:
62:08:00 SRC=192.168.10.11 DST=192.168.10.12 LEN=40 TOS=0x00 PREC=0x00 TTL=5
1 ID=12615 PROTO=TCP SPT=64614 DPT=2288 WINDOW=1024 RES=0x00 URGP=0
[68892.118567] MALFORMED: IN=eth0 OUT= MAC=ba:7b:37:f3:10:b6:22:fa:a1:9c:78:
62:08:00 SRC=192.168.10.11 DST=192.168.10.12 LEN=40 TOS=0x00 PREC=0x00 TTL=3
8 ID=49220 PROTO=TCP SPT=64614 DPT=5000 WINDOW=1024 RES=0x00 URGP=0
[68892.118606] MALFORMED: IN=eth0 OUT= MAC=ba:7b:37:f3:10:b6:22:fa:a1:9c:78:
62:08:00 SRC=192.168.10.11 DST=192.168.10.12 LEN=40 TOS=0x00 PREC=0x00 TTL=4
1 ID=41175 PROTO=TCP SPT=64614 DPT=765 WINDOW=1024 RES=0x00 URGP=0
[68892.118625] MALFORMED: IN=eth0 OUT= MAC=ba:7b:37:f3:10:b6:22:fa:a1:9c:78:
62:08:00 SRC=192.168.10.11 DST=192.168.10.12 LEN=40 TOS=0x00 PREC=0x00 TTL=5
8 ID=36571 PROTO=TCP SPT=64614 DPT=13782 WINDOW=1024 RES=0x00 URGP=0
```

Test D: Spoofed Packet Rejection

Screenshots:

- [Insert screenshot of `hping3` sending packets with reserved source IPs]

```
┌──(kevin㉿kali)-[~/Desktop]
└─$ sudo hping3 -S -p 80 --spoof 10.0.0.2 192.168.10.12
HPING 192.168.10.12 (eth0 192.168.10.12): S set, 40 headers + 0 data bytes
xXXXXXXXXXXX
```

- [Insert screenshot of `hping3` sending a packet with VM-D's own IP as source]

```
┌──(kevin㉿kali)-[~/Desktop]
└─$ sudo hping3 -S -p 80 --spoof 192.168.10.11 192.168.10.12
HPING 192.168.10.12 (eth0 192.168.10.12): S set, 40 headers + 0 data bytes
len=44 ip=192.168.10.12 ttl=64 DF id=0 sport=80 flags=SA seq=0 win=64240 rtt=7.8 ms
len=44 ip=192.168.10.12 ttl=64 DF id=0 sport=80 flags=SA seq=1 win=64240 rtt=6.8 ms
len=44 ip=192.168.10.12 ttl=64 DF id=0 sport=80 flags=SA seq=2 win=64240 rtt=6.4 ms
len=44 ip=192.168.10.12 ttl=64 DF id=0 sport=80 flags=SA seq=3 win=64240 rtt=9.3 ms
len=44 ip=192.168.10.12 ttl=64 DF id=0 sport=80 flags=SA seq=4 win=64240 rtt=8.6 ms
len=44 ip=192.168.10.12 ttl=64 DF id=0 sport=80 flags=SA seq=5 win=64240 rtt=5.4 ms
```

- [Insert screenshot of `dmesg` showing `MALFORMED` and `ILLEGAL-IP/PORT` log entries]

```
┌──(kevin㉿kali)-[~/Documents/assignment3]
└─$ sudo dmesg | tail -10
[68892.326673] MALFORMED: IN=eth0 OUT= MAC=ba:7b:37:f3:10:b6:22:fa:a1:9c:78:
62:08:00 SRC=192.168.10.11 DST=192.168.10.12 LEN=40 TOS=0x00 PREC=0x00 TTL=5
0 ID=14817 PROTO=TCP SPT=64614 DPT=1074 WINDOW=1024 RES=0x00 URGP=0
[68892.326859] MALFORMED: IN=eth0 OUT= MAC=ba:7b:37:f3:10:b6:22:fa:a1:9c:78:
62:08:00 SRC=192.168.10.11 DST=192.168.10.12 LEN=40 TOS=0x00 PREC=0x00 TTL=4
4 ID=63333 PROTO=TCP SPT=64614 DPT=3404 WINDOW=1024 RES=0x00 URGP=0
[68892.327139] MALFORMED: IN=eth0 OUT= MAC=ba:7b:37:f3:10:b6:22:fa:a1:9c:78:
62:08:00 SRC=192.168.10.11 DST=192.168.10.12 LEN=40 TOS=0x00 PREC=0x00 TTL=5
3 ID=45202 PROTO=TCP SPT=64614 DPT=1186 WINDOW=1024 RES=0x00 URGP=0
[68892.327235] MALFORMED: IN=eth0 OUT= MAC=ba:7b:37:f3:10:b6:22:fa:a1:9c:78:
62:08:00 SRC=192.168.10.11 DST=192.168.10.12 LEN=40 TOS=0x00 PREC=0x00 TTL=4
6 ID=24094 PROTO=TCP SPT=64614 DPT=1334 WINDOW=1024 RES=0x00 URGP=0
[68892.327323] MALFORMED: IN=eth0 OUT= MAC=ba:7b:37:f3:10:b6:22:fa:a1:9c:78:
62:08:00 SRC=192.168.10.11 DST=192.168.10.12 LEN=40 TOS=0x00 PREC=0x00 TTL=3
9 ID=64508 PROTO=TCP SPT=64614 DPT=55056 WINDOW=1024 RES=0x00 URGP=0
[68892.327400] MALFORMED: IN=eth0 OUT= MAC=ba:7b:37:f3:10:b6:22:fa:a1:9c:78:
62:08:00 SRC=192.168.10.11 DST=192.168.10.12 LEN=40 TOS=0x00 PREC=0x00 TTL=4
8 ID=61112 PROTO=TCP SPT=64614 DPT=32768 WINDOW=1024 RES=0x00 URGP=0
```

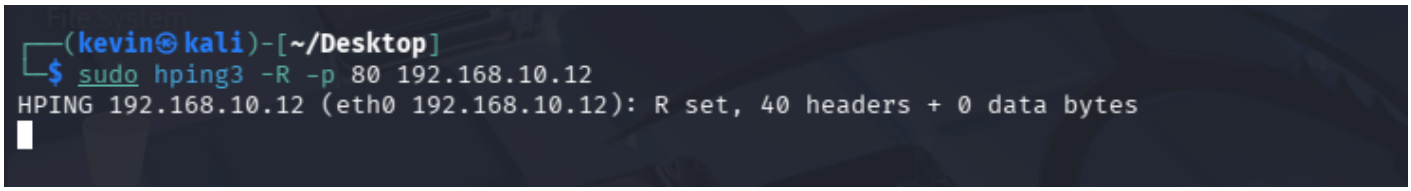
Test E: Invalid State Packet Handling

Screenshots:

- [Insert screenshot of `hping3` sending ACK-only packets without a prior session]

```
┌──(kevin㉿kali)-[~/Desktop]
└─$ sudo hping3 -A -p 80 192.168.10.12
HPING 192.168.10.12 (eth0 192.168.10.12): A set, 40 headers + 0 data bytes
```

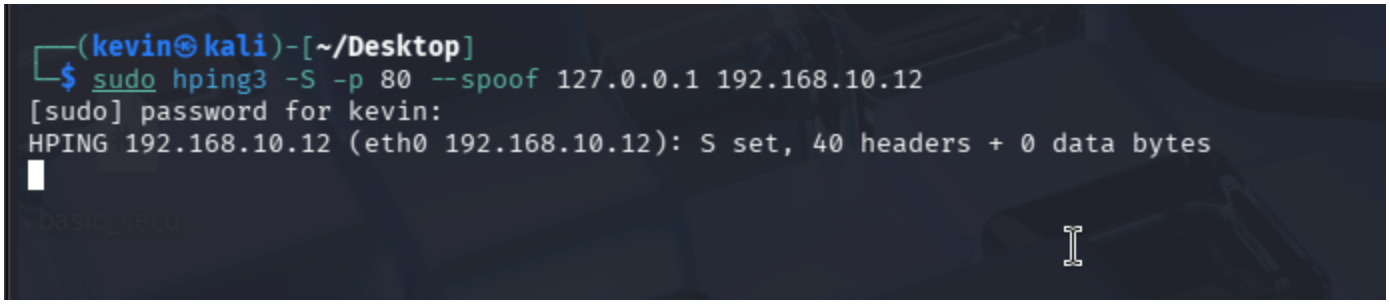
- [Insert screenshot of `dmesg` confirming INVALID state drops]



Test F: Loopback Enforcement

Screenshots:

- [Insert screenshot of `hping3` from VM-A crafting a packet with source IP `127.0.0.1`]



- [Insert screenshot of `dmesg` confirming the packet was dropped]

```
└─$ sudo dmesg | tail -10
[sudo] password for kevin:
[78947.323022] DROP-GENERIC: IN=eth0 OUT= MAC=ff:ff:ff:ff:ff:ff:ce:83:cf:61:
c8:ba:08:00 SRC=192.168.10.13 DST=192.168.10.255 LEN=272 TOS=0×00 PREC=0×00
TTL=64 ID=0 DF PROTO=UDP SPT=138 DPT=138 LEN=252
[78947.323576] DROP-GENERIC: IN=eth0 OUT= MAC=ff:ff:ff:ff:ff:ff:ce:83:cf:61:
c8:ba:08:00 SRC=192.168.10.13 DST=192.168.10.255 LEN=243 TOS=0×00 PREC=0×00
TTL=64 ID=0 DF PROTO=UDP SPT=138 DPT=138 LEN=223
[79667.313997] DROP-GENERIC: IN=eth0 OUT= MAC=ff:ff:ff:ff:ff:ff:ce:83:cf:61:
c8:ba:08:00 SRC=192.168.10.13 DST=192.168.10.255 LEN=272 TOS=0×00 PREC=0×00
TTL=64 ID=0 DF PROTO=UDP SPT=138 DPT=138 LEN=252
[79667.314328] DROP-GENERIC: IN=eth0 OUT= MAC=ff:ff:ff:ff:ff:ff:ce:83:cf:61:
c8:ba:08:00 SRC=192.168.10.13 DST=192.168.10.255 LEN=243 TOS=0×00 PREC=0×00
TTL=64 ID=0 DF PROTO=UDP SPT=138 DPT=138 LEN=223
[80387.300386] DROP-GENERIC: IN=eth0 OUT= MAC=ff:ff:ff:ff:ff:ff:ce:83:cf:61:
c8:ba:08:00 SRC=192.168.10.13 DST=192.168.10.255 LEN=272 TOS=0×00 PREC=0×00
TTL=64 ID=0 DF PROTO=UDP SPT=138 DPT=138 LEN=252
[80387.300866] DROP-GENERIC: IN=eth0 OUT= MAC=ff:ff:ff:ff:ff:ff:ce:83:cf:61:
c8:ba:08:00 SRC=192.168.10.13 DST=192.168.10.255 LEN=243 TOS=0×00 PREC=0×00
TTL=64 ID=0 DF PROTO=UDP SPT=138 DPT=138 LEN=223
[81107.289061] DROP-GENERIC: IN=eth0 OUT= MAC=ff:ff:ff:ff:ff:ff:ce:83:cf:61:
c8:ba:08:00 SRC=192.168.10.13 DST=192.168.10.255 LEN=272 TOS=0×00 PREC=0×00
TTL=64 ID=0 DF PROTO=UDP SPT=138 DPT=138 LEN=252
[81107.289268] DROP-GENERIC: IN=eth0 OUT= MAC=ff:ff:ff:ff:ff:ff:ce:83:cf:61:
c8:ba:08:00 SRC=192.168.10.13 DST=192.168.10.255 LEN=243 TOS=0×00 PREC=0×00
TTL=64 ID=0 DF PROTO=UDP SPT=138 DPT=138 LEN=223
[81823.507494] DROP-GENERIC: IN=eth0 OUT= MAC=ff:ff:ff:ff:ff:ff:ce:83:cf:61:
c8:ba:08:00 SRC=192.168.10.13 DST=192.168.10.255 LEN=272 TOS=0×00 PREC=0×00
TTL=64 ID=0 DF PROTO=UDP SPT=138 DPT=138 LEN=252
[81823.507815] DROP-GENERIC: IN=eth0 OUT= MAC=ff:ff:ff:ff:ff:ff:ce:83:cf:61:
c8:ba:08:00 SRC=192.168.10.13 DST=192.168.10.255 LEN=243 TOS=0×00 PREC=0×00
TTL=64 ID=0 DF PROTO=UDP SPT=138 DPT=138 LEN=223

┌──(kevin㉿kali)-[~/Documents/assignment3]
```

Test G: UDP Behavior

Screenshots:

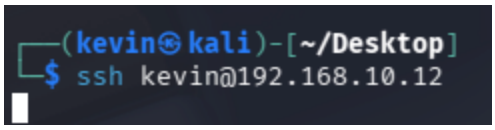
- [Insert screenshot of UDP packets sent from VM-A to VM-D]

```
┌──(kevin㉿kali)-[~/Desktop]
└─$ sudo hping3 --udp -p 9999 192.168.10.12
HPING 192.168.10.12 (eth0 192.168.10.12): udp mode set, 28 headers + 0 data bytes
```

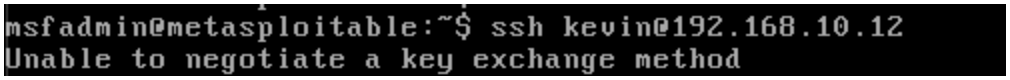
Test H: SSH Access Control

Screenshots:

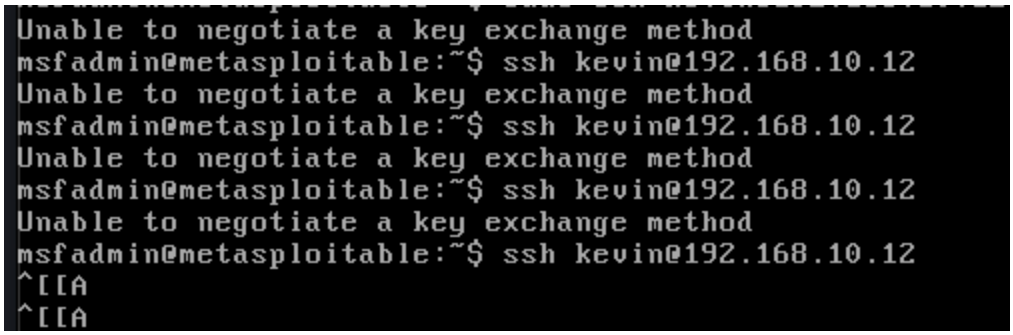
- [Insert screenshot of SSH attempt from VM-A (untrusted, blocked)]



- [Insert screenshot of SSH from VM-T (trusted, allowed)]



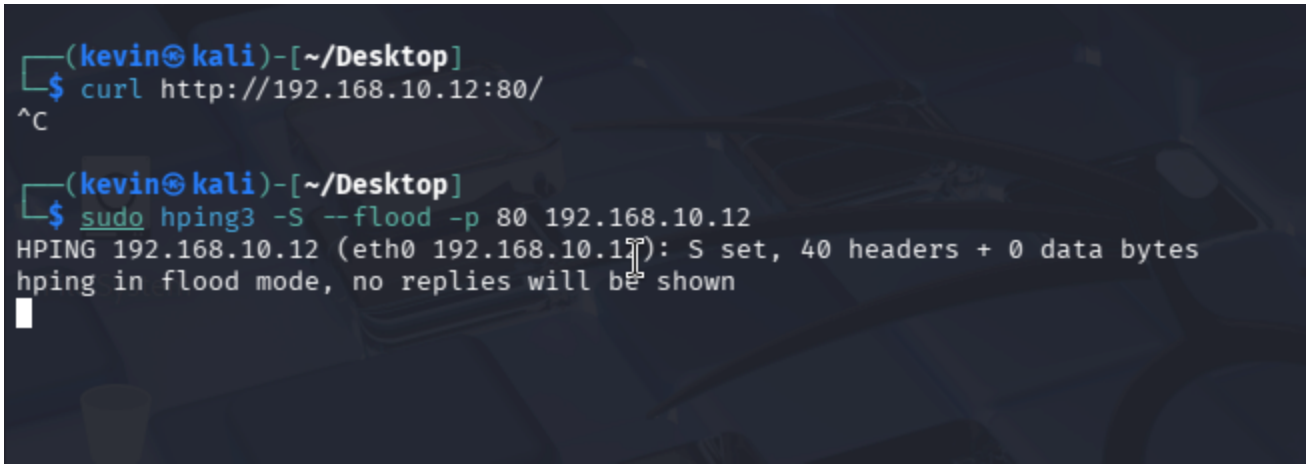
- [Insert screenshot of SSH rate-limit triggered after three quick attempts from VM-T]



Test I: HTTP Access and Flood Protection

Screenshots:

- [Insert screenshot of `curl http://<VM-D-IP>:80/` from VM-A succeeding]



Analysis

The ruleset follows a default-deny approach where everything is blocked unless explicitly allowed. This made the most sense for a server that only needs to serve HTTP and accept SSH from one trusted machine.

The base rules handle the obvious stuff first — loopback is always allowed, INVALID state packets are dropped immediately, and ESTABLISHED/RELATED connections are accepted early so that outbound replies like DNS responses aren't blocked on the way back in.

The anti-spoofing rules drop three categories of obviously fake traffic: packets claiming to be from the loopback range but not arriving on lo, packets using VM-D's own IP as their source, and anything coming from outside the 192.168.10.0/24 subnet. These all get logged with MALFORMED or ILLEGAL-IP/PORT prefixes depending on the type of violation.

Scan detection covers the classic nmap flag combinations — NULL, Xmas, SYN-FIN, and ACK-only on new connections. These flag combinations don't appear in legitimate traffic so they can be dropped and logged as SCAN-TCP without any risk of false positives.

The SYN flood chain runs before the service rules. Anything within the 5/s rate limit gets passed back to the INPUT chain for normal processing, while anything over the limit is logged as DOS-RATE-LIMIT and dropped. This way flood protection doesn't interfere with legitimate traffic getting through to HTTP and SSH.

SSH is locked down to VM-T only with a rate limit of 3 new connections per minute. HTTP allows any internal host but caps concurrent sessions at 20 per IP and new connections at 10/s. The DROP-GENERIC rule at the end catches anything that slipped through, which was mostly UDP traffic during testing.

Log Analysis:

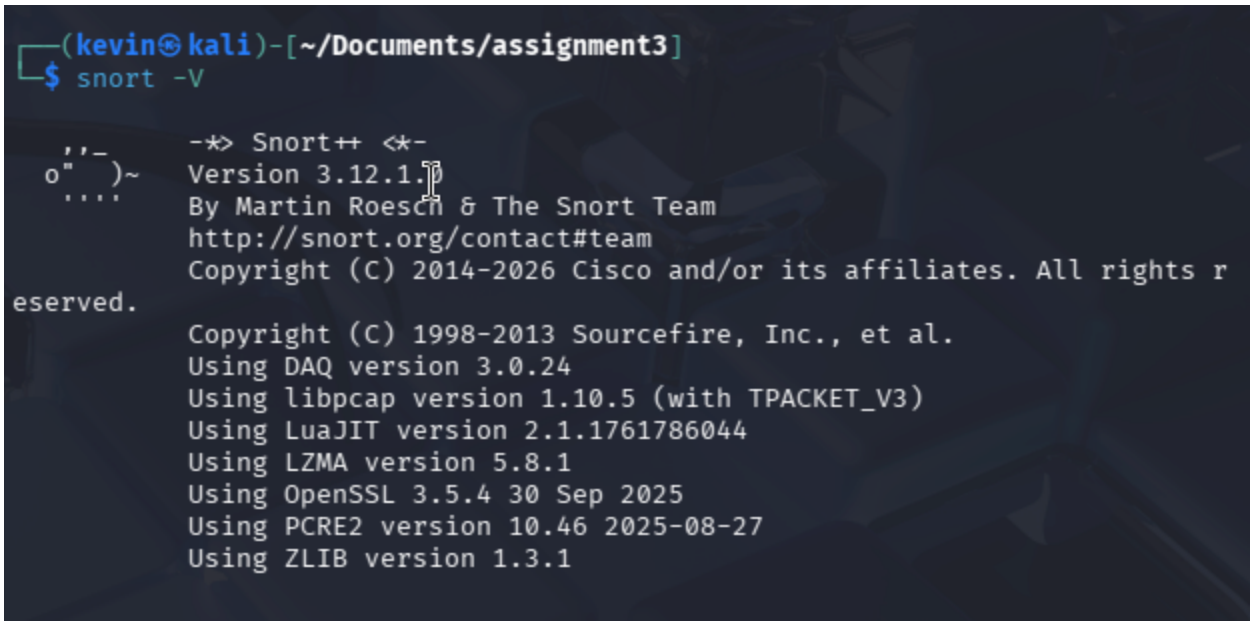
The dmesg output confirmed everything worked as expected. SCAN-TCP showed up for the FIN, Xmas, and NULL scans. MALFORMED caught the spoofed loopback and own-IP packets. ILLEGAL-IP/PORT fired for the SSH attempt from VM-A. DOS-RATE-LIMIT appeared during the SYN and HTTP floods. DROP-GENERIC picked up the UDP test packets. Filtering by prefix with grep made it easy to isolate each category during testing.

Part 3: Web Application Intrusion Detection with Snort

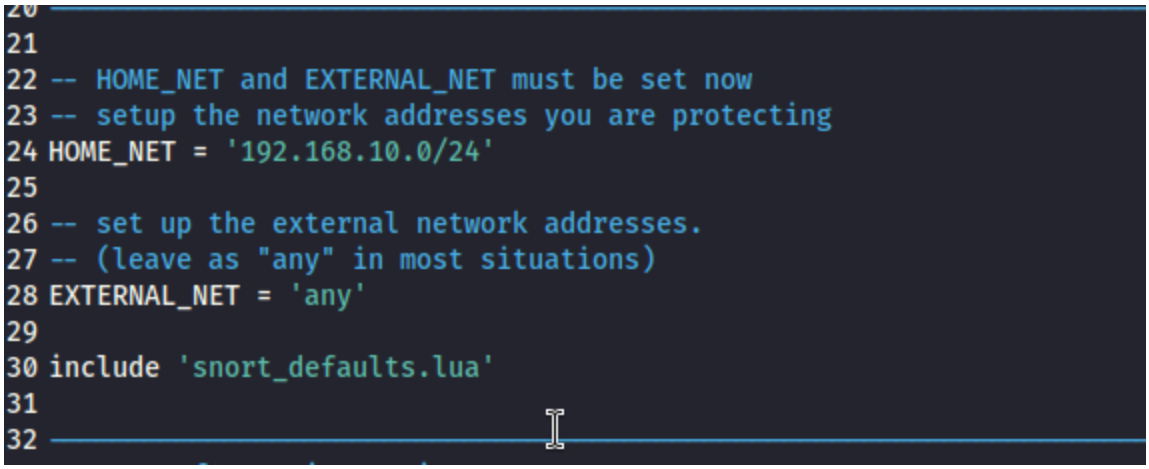
Task 1: Snort Configuration

Screenshots:

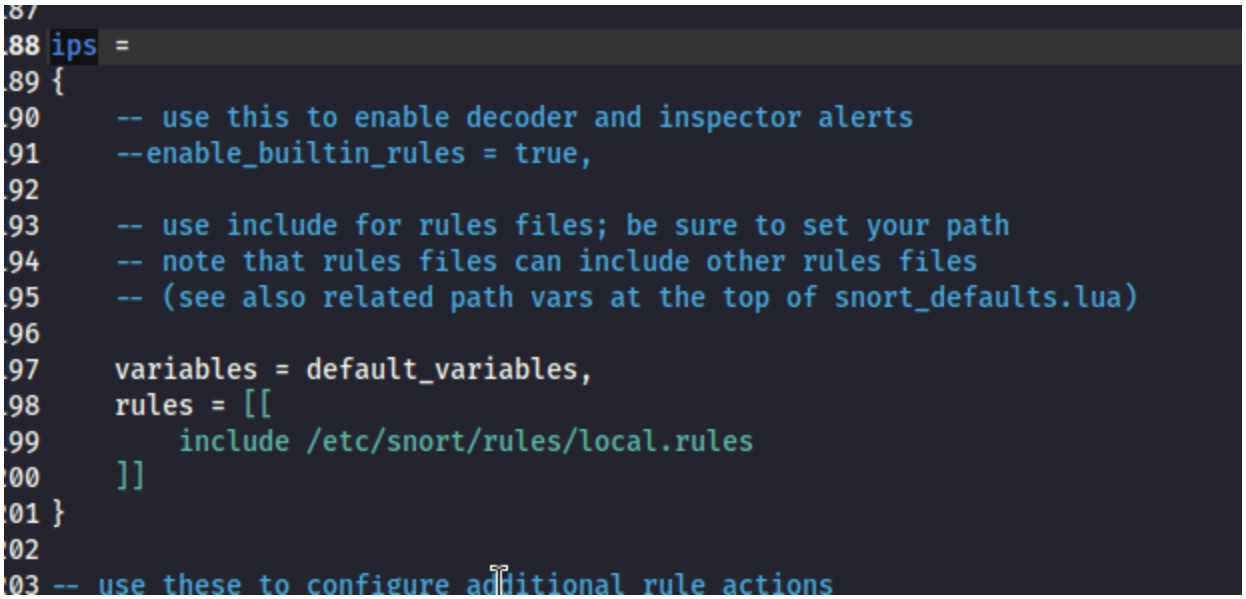
- [Insert screenshot of `snort -V` output confirming version]



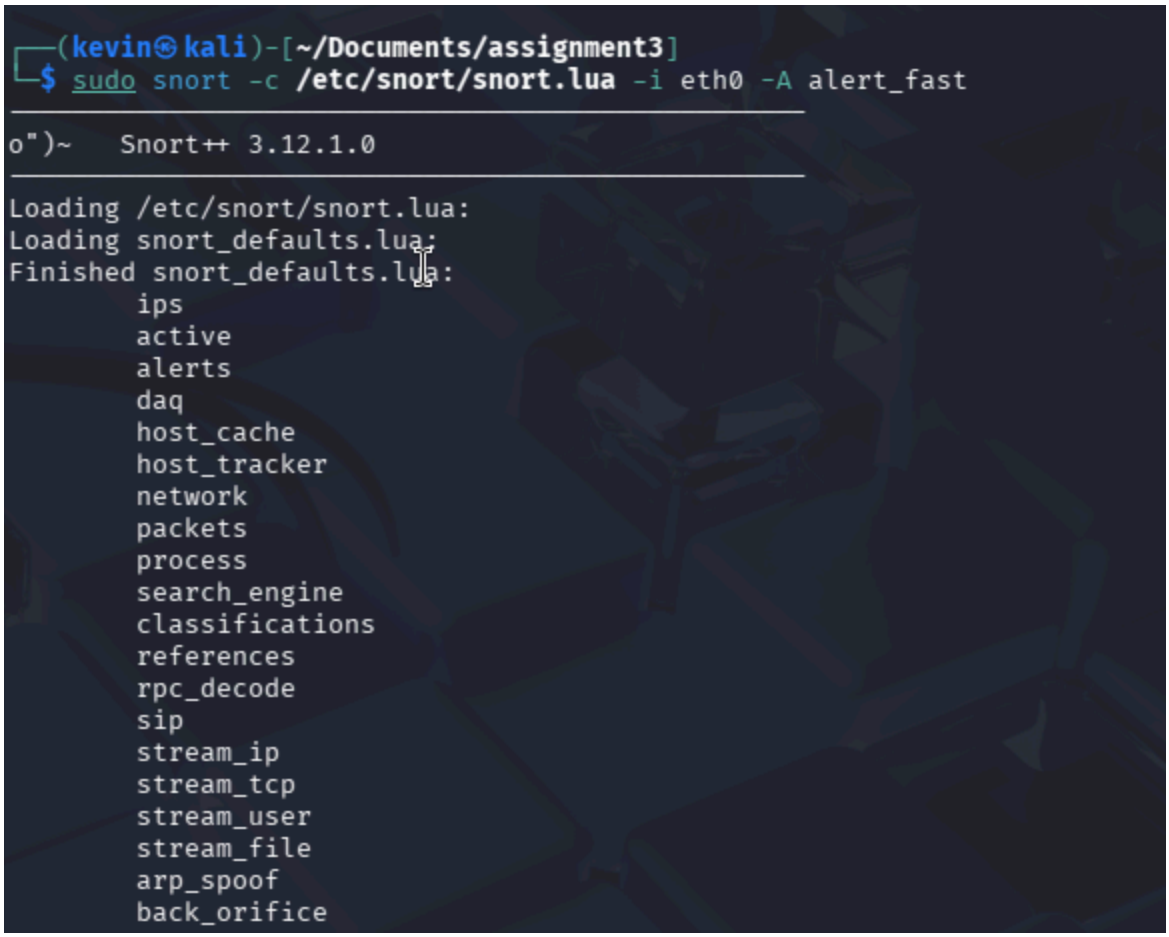
- [Insert screenshot of the modified `HOME_NET` and `EXTERNAL_NET` lines in `snort.lua`]



- [Insert screenshot of the `ips` block in `snort.lua` showing `local.rules` included]



- [Insert screenshot of Snort startup output showing rules loaded and listening on eth0]

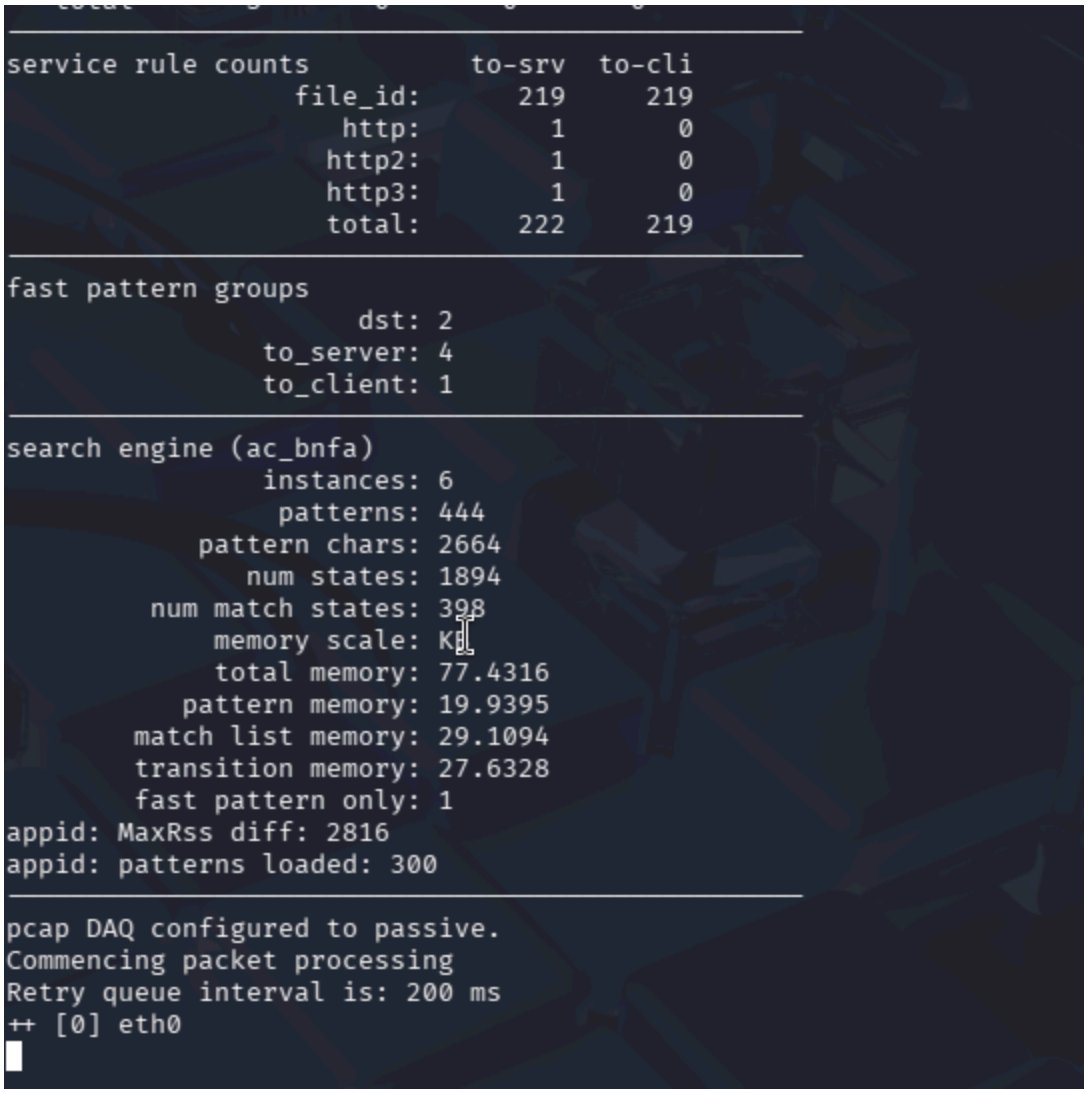


Task 2: Attack Detection

Baseline: No Attack

Screenshots:

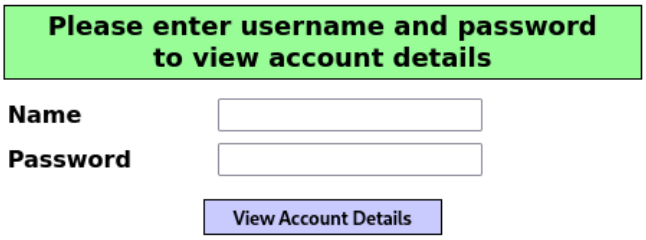
- [Insert screenshot of normal `curl` from VM-A with no Snort alerts on VM-D]



Attack 1: SQL Injection (sid:200001)

Screenshots:

- [Insert screenshot of the SQL injection payload submitted in DVWA on VM-A]



Results for . 17 records found.

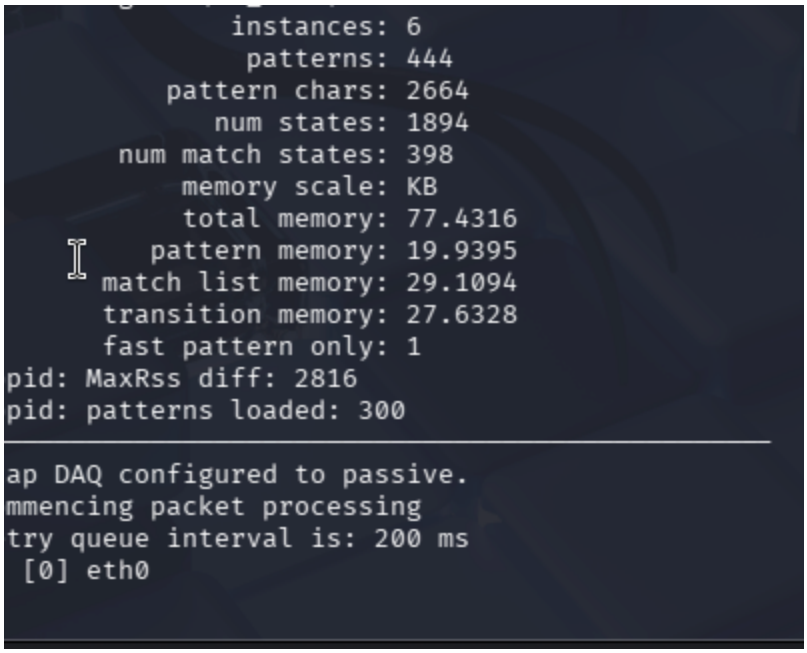
Username=admin
Password=adminpass
Signature=Monkey!

Username=adrian
Password=somepassword
Signature=Zombie Films Rock!

Username=john
Password=monkey
Signature=I like the smell of confunk

Username=jeremy
Password=password
Signature=d1373 1337 speak

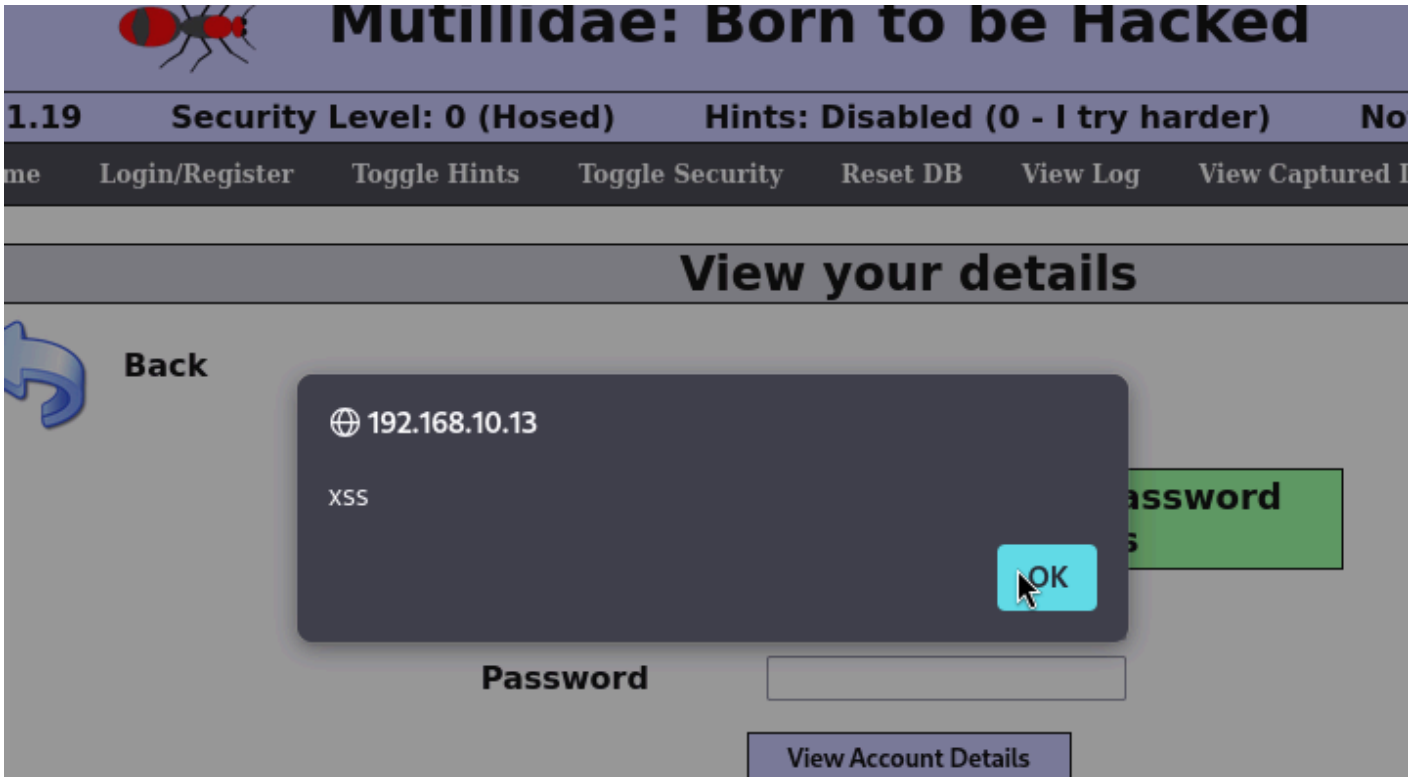
- [Insert screenshot of Snort `alert_fast` output showing the SQLi alert on VM-D]



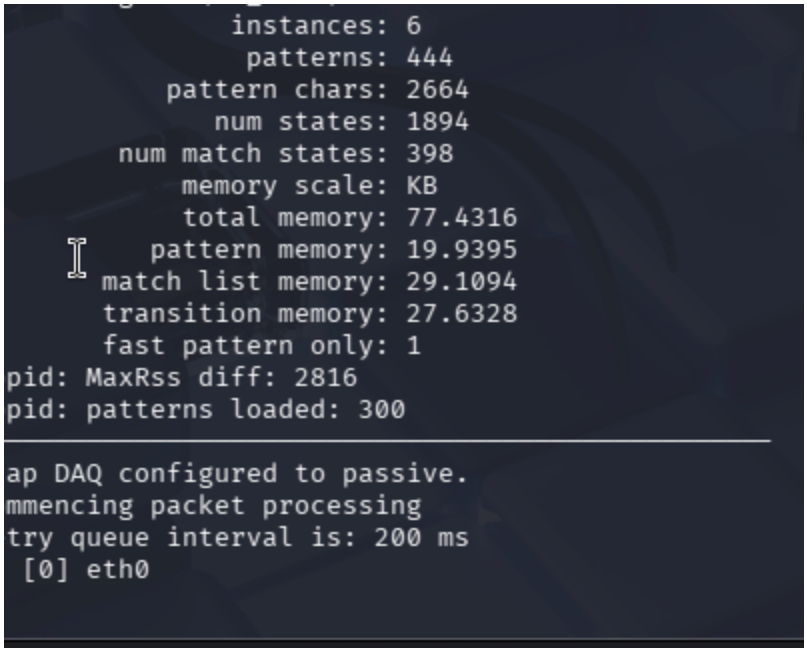
Attack 2: XSS (sid:200002)

Screenshots:

- [Insert screenshot of the XSS payload submitted on VM-A]



- [Insert screenshot of Snort `alert_fast` output showing the XSS alert on VM-D]



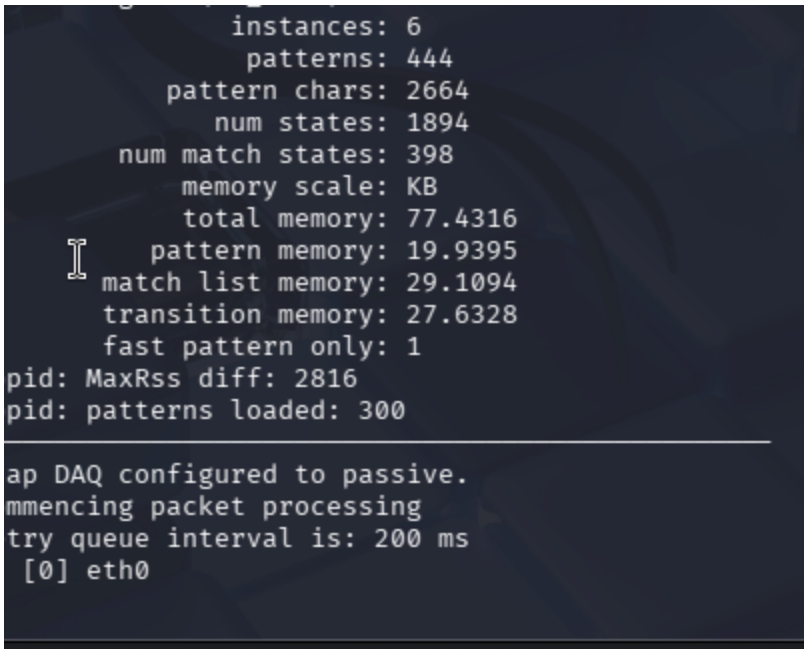
Attack 3: LFI (sid:200003)

Screenshots:

- [Insert screenshot of the LFI URL (`../../../../etc/passwd`) in the browser on VM-A]

alt text

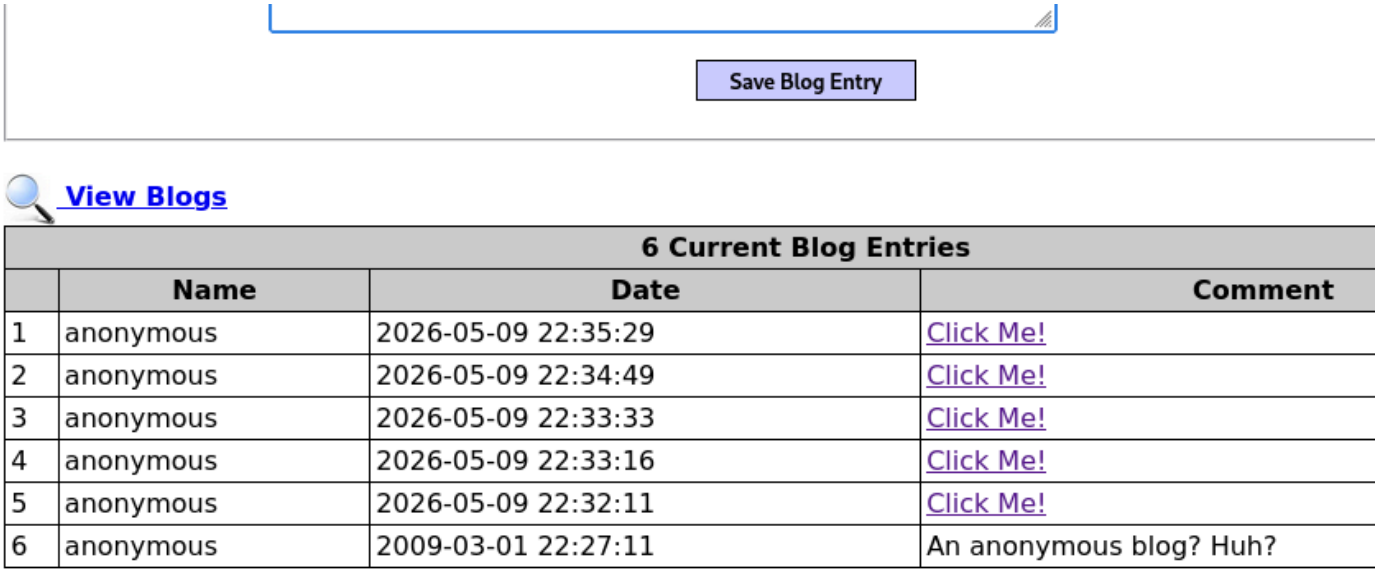
- [Insert screenshot of Snort `alert_fast` output showing the LFI alert on VM-D]

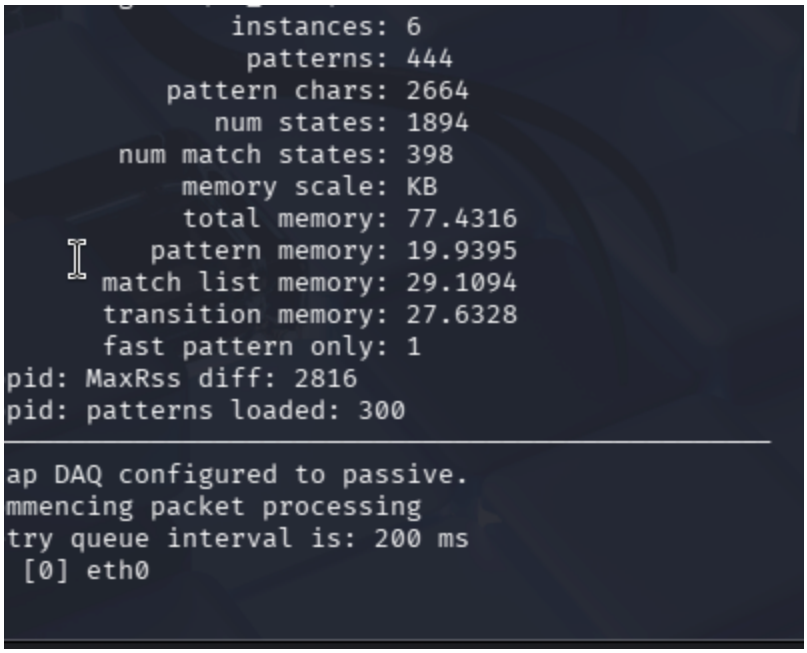


Attack 4: CSRF (sid:200004)

Screenshots:

- [Insert screenshot of `csrf_attack.html` and the Python server serving it on VM-A]




instances: 6
patterns: 444
pattern chars: 2664
num states: 1894
num match states: 398
memory scale: KB
total memory: 77.4316
pattern memory: 19.9395
match list memory: 29.1094
transition memory: 27.6328
fast pattern only: 1
pid: MaxRss diff: 2816
pid: patterns loaded: 300
ap DAQ configured to passive.
mmencing packet processing
try queue interval is: 200 ms
[0] eth0